



## Tarefa 2

### Expansão e Complexidade

MC322 - Programação Orientada a Objetos

**Professor:** Marcelo da Silva Reis

**PEDs:** Athyrson Ribeiro, Matheus Lozano

**PADs:** João Pedro Frare

## 1. Descrição Geral

Uma planta industrial automatizada é composta por diversos equipamentos responsáveis por transformar matérias-primas em produtos acabados. Na Tarefa 1, desenvolvemos uma versão simplificada com uma linha de produção básica. Agora, na **Tarefa 2**, evoluiremos este sistema com **herança**, **polimorfismo**, **coleções** e **aleatoriedade**.

### 1.1. Evolução da Planta

A planta agora deve:

- Produzir diferentes tipos de produtos usando **herança**;
- Gerenciar demandas de produtos finais;
- Simular **falhas de equipamentos**;
- Controlar um **budget** alocado;
- Manter um **armazém** de produtos fabricados.

### 1.2. Novos Conceitos

1. **Herança em Produtos:** Classe abstrata **Produto** com 3 subclasses;
2. **Herança em Máquinas:** Classe abstrata **Maquina** com 3 subclasses;
3. **Coleções:** **ArrayList** para filas, demandas e armazém;
4. **Aleatoriedade:** **Random** para falhas;
5. **Métodos Estáticos e Privados:** Contadores e lógica interna.

## 2. Objetivos

- Estender conhecimentos de POO: herança, classes abstratas, encapsulamento;
- Aplicar **ArrayList** para coleções de dados;
- Utilizar **Random** para aleatoriedade;
- Implementar métodos privados e estáticos;
- Evoluir a planta da Tarefa 1;
- Criar hierarquia de produtos com qualidades variadas;
- Criar hierarquia de máquinas com diferentes probabilidades de falha;
- Gerenciar budget e demandas de produtos finais.

### 3. Atividades

#### 3.1. Implementação das Classes

##### 3.1.1. Classe Abstrata Produto

Representa um produto genérico. Esta classe **deve ser abstrata**.

##### Atributos

- `id` – Identificador único;
- `nome` – Nome do produto;
- `status` – Estado atual;
- `quantidadeMateriaPrimaPorUnidade` – Matéria-prima necessária por unidade;
- `qualidade` – Qualidade do produto (0.0 a 1.0);
- `probabilidadeFalhaAcumulada` – Acumula a chance de falha ao passar pelas máquinas;
- `totalProdutosFabricados` – Atributo **estático** para contabilizar instâncias geradas.

##### Métodos Abstratos

- `processar` – Define o processamento específico;
- `calcularTempoProducao` – Tempo de produção;
- `getTipo` – Retorna o tipo do produto.

##### Métodos Concretos

- `getId`, `getNome`, `getStatus`, `setStatus`;
- `getQuantidadeMateriaPrimaPorUnidade`;
- `aumentarProbabilidadeFalha`;
- `getQualidade`.

##### 3.1.2. Subclasses de Produto

**Importante:** Os alunos **devem substituir os nomes** das subclasses por nomes coerentes com o tema da fábrica escolhido. Os nomes abaixo são apenas exemplos genéricos.

- `ProdutoAltaQualidade` – Qualidade 0.9, maior demanda de matéria-prima;
- `ProdutoMediaQualidade` – Qualidade 0.7, demanda média de matéria-prima;
- `ProdutoBaixaQualidade` – Qualidade 0.5, menor demanda de matéria-prima;

**Relação Qualidade × Inspeção** A qualidade do produto deve ser **diretamente proporcional** à chance de falha na inspeção:

- Produtos de **alta qualidade** têm **maior chance** de serem rejeitados na inspeção, pois os critérios são mais rigorosos;
- Produtos de **baixa qualidade** têm **menor chance** de serem rejeitados, pois os critérios são mais flexíveis.

##### 3.1.3. Classe Abstrata Maquina

Representa uma máquina genérica. Esta classe **deve ser abstrata**.

## Atributos

- nome – Nome da máquina;
- ligada – Estado atual;
- capacidadeMaxima – Capacidade de processamento;
- probabilidadeFalha – Chance de falha;
- custoOperacao – Custo por operação.

## Métodos Abstratos

- processar – Processa um produto;
- getTipo – Retorna o tipo da máquina.

## Métodos Concretos

- ligar, desligar, estaLigada;
- getNome, getCustoOperacao;
- verificarFalha – Método protegido com aleatoriedade.

### 3.1.4. Subclasses de Maquina

**Importante:** Os alunos **devem substituir os nomes** das subclasses por nomes coerentes com o tema da fábrica escolhido. Os nomes abaixo são apenas exemplos genéricos.

Devem ser implementadas pelo menos **3 subclasses** de Maquina:

- MaquinaProcessamento – Não falha diretamente, mas tem **X % de chance** de aumentar a probabilidade de falha do produto processado;
- MaquinaEmbalagem – Não falha diretamente, mas tem **Y % de chance** de aumentar a probabilidade de falha do produto embalado;
- MaquinaInspecao – **Pode falhar** com probabilidade de **Z %**, resultando em uma inspeção incorreta.

## Comportamento das Máquinas

- As máquinas de **processamento** e **embalagem** não falham diretamente;
- Elas podem **aumentar a probabilidade de falha acumulada** do produto que passa por elas;
- A máquina de **inspeção** é a única que pode falhar diretamente;
- A probabilidade de falha do produto é acumulativa: quanto mais máquinas o produto passar, maior a chance de falha.

## Exemplo de Fluxo

Produto entra na máquina de processamento  
↓  
Chance de X% de aumentar probabilidade de falha do produto  
↓  
Produto entra na máquina de embalagem  
↓  
Chance de Y% de aumentar probabilidade de falha do produto  
↓  
Produto entra na máquina de inspeção  
↓  
Chance base de Z% de falha na inspeção  
↓  
Se o produto acumulou alta probabilidade de falha,  
maior chance de ser rejeitado na inspeção

### 3.1.5. Classe MateriaPrima

#### Atributos

- id, nome, quantidade, unidade;
- custoPorUnidade.

#### Métodos

- consumir, adicionarEstoque;
- verificarDisponibilidade.

### 3.1.6. Classe Demanda

#### Atributos

- tipoProduto – String com o tipo ou nome do produto demandado;
- quantidadeProdutos – Quantidade de produtos finais;
- atendida – Status da demanda.

#### Métodos

- atualizarQuantidade;
- calcularMateriaPrimaNecessaria;
- atender.

### 3.1.7. Classe GerenciadorProducao

#### Atributos

- demandas – ArrayList de demandas;
- produtosFabricados – ArrayList do armazém;
- maquinas – ArrayList de máquinas;
- materiaPrima – Matéria-prima;
- budget – Orçamento disponível.

#### Métodos

- registrarDemanda, atualizarDemanda;
- fabricarDemanda;
- comprarMateriaPrima;
- exibirBudget, exibirArmazem;
- calcularCustoProducao – Método privado.

### 3.2. Menu do Sistema

O menu deve ser **claro e organizado**, com seções bem definidas. Exemplo:

FÁBRICA DE PÃES ARTESANAIS

BUDGET ATUAL: R\$1000.00

ATUALIZAR DEMANDAS

- 1 - Atualizar demanda de Pão Artesanal
- 2 - Atualizar demanda de Pão de Forma
- 3 - Atualizar demanda de Pão Simples

FABRICAR

- 4 - Fabricar Pão Artesanal
- 5 - Fabricar Pão de Forma
- 6 - Fabricar Pão Simples

CONSULTAR

- 7 - Ver armazém
- 8 - Ver estoque de matéria-prima

COMPRAR MATÉRIA-PRIMA

- 9 - Comprar matéria-prima

0 - SAIR

ESCOLHA:

### 3.3. Pontos de Atenção

- Atributos privados com métodos públicos;
- Produto e Maquina devem ser abstratas;
- Nomes das subclasses de Produto devem ser coerentes com o tema;
- Qualidade diretamente proporcional à chance de falha na inspeção;
- ArrayList para coleções;
- Random para falhas;
- Métodos estáticos para contadores (ex: total de produtos fabricados na classe Produto);
- Métodos privados para lógica interna;
- Apenas entradas numéricas;
- O Budget é debitado em duas ocasiões: ao comprar matéria-prima do fornecedor e ao pagar o custo de operação de cada máquina processada;
- Matéria-prima consumida conforme demanda;
- Armazém armazena produtos fabricados;
- Sem gestão de lucro/prejuízo.

## 4. Avaliação

### 1. Modelagem e Implementação (2,5 pontos)

- Classe abstrata **Produto** com 3 subclasses (nomes coerentes com o tema);
- Classe abstrata **Maquina** com 3 subclasses;
- Herança e polimorfismo corretos;
- Relação qualidade  $\times$  inspeção implementada (diretamente proporcional);
- Encapsulamento adequado;
- Métodos estáticos e privados.

### 2. Funcionamento (2,5 pontos)

- Menu claro e organizado;
- Gerenciamento de demandas;
- Simulação de falhas;
- Controle de budget;
- Atualização de estoque e armazém;
- Validação de entradas.

### 3. Organização do Código (2,0 pontos)

- Cada classe em arquivo próprio;
- Código legível;
- Nomes adequados;
- Separação de responsabilidades.

### 4. Criatividade (3,0 pontos)

- Evolução do tema da Tarefa 1;
- Nomes criativos para subclasses de produtos;
- Tela de introdução atualizada;
- `justificativa.txt` atualizado;
- Toque humano e originalidade.

### 5. Penalidade: Não compilação = -5,0 pontos.

## 5. Entrega

1. **GitHub:** Release `tarefa2-RA1-RA2` com `justificativa.txt`;
2. **Google Forms:** Nomes, RAs e link da release.

O prazo de submissão é o dia **10/09/2026 às 23h59**. Entregas atrasadas serão penalizadas em **25 % da nota obtida**. Não serão aceitas entregas atrasadas após **17/09/2026**.

### 5.1. Compilação e Execução

```
javac -d bin $(find src -name "*.java")
java -cp bin Main
```

## 6. Dicas de Boas Práticas

- **Construtores:** Lembre-se de utilizar `super()` nos construtores das subclasses para inicializar corretamente os atributos da classe mãe.
- **DRY:** Use herança para evitar duplicação;
- **KISS:** Soluções simples e diretas;
- **YAGNI:** Não implemente o que não é necessário.

## 6.1. Considerações Específicas

- **Herança:** Produto e Máquina abstratas;
- **Nomes das Subclasses:** Coerentes com o tema da fábrica;
- **Qualidade × Inspeção:** Diretamente proporcionais;
- **Aleatoriedade:** Probabilidades e falhas;
- **Coleções:** ArrayList para gestão;
- **Métodos Estáticos:** Contadores;
- **Métodos Privados:** Cálculos internos;
- **Budget:** Alocado na Main, gerenciado pela GerenciadorProducao;
- **Demanda:** Quantidade de produtos finais (entrada do usuário).

Lembre-se: o objetivo é evoluir a planta com conceitos avançados de POO!